The tools and techniques of R programming Lesson-8 Dealing with system-defined functions and formatting in R

By
Prof. Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
For any query,Email-Id:hod.ca@technoindiaeducation.com

October 29, 2023

Dealing with some useful functions

Defining a new line: It can be done easily by using the **cat()** function as follows:

cat("\n")

Clearing the screen: Sometimes, we need to clear the undesirable screen displays before showing the output of a program. There is no ready-made function available in R for this purpose. This can be done interactively by pressing **CTRL+ L** key together. However, we need to do this very often in programs. It is specially required in R programs because, R scripts are interpreted for execution; so, all the R codes are seen on the screen that looks clumsy. So, we need some method for this purpose. The purpose can be served by printing a newline a large number of times and a function can be developed with this code. Here is the code:

cls=function() cat(rep("\n",30))

Here, **rep()** is the function for replication of something.

Now, the code: **cls()** can be used to clear the screen after execution of this function once.

Dealing with some useful functions–Contd.

The following function can be used to keep the system in a waiting state while running some code:

```
library(tcltk)
 mywait = function() {
tt = tktoplevel()
 tkpack( tkbutton(tt, text='Continue', command=function()tkdestroy(tt)),
 side='bottom')
 tkbind(tt,'<Key>', function()tkdestroy(tt) )
 tkwait.window(tt)
 }
```

Dealing with termination in between execution

It is sometimes required to terminate an R script when some invalid condition arises. R does not provide us with any foolproof function for the purpose. We show here a function that may be included in a program for this purpose.

```
exit = function() {
 .Internal(.invokeRestart(list(NULL, NULL), NULL))
}
```

Now, if **this function can be called as exit()**, it will serve the desired purpose.

Dealing with some useful functions–Contd.

Formatting data for pretty printing

Most often, we like to print output data values in a desirable form to make it better readable, better understandable and visually pleasing. The finishing touch that we provide on the information to give it a desirable look is called **formatting**. There is an R function for this purpose and it is called the **format()** function. The syntax of the function has been discussed below:

format(x, trim=FALSE,digits=NULL,nsmall=0L,justify=c("left", "right","centre","none"), width=NULL, na.encode=TRUE, scientific=NA, big.mark=" ", big.interval=3L, small.mark=" ", small.interval=5L, decimal.mark= getOption("OutDec"),zero.print=NULL, drop0trailing=FALSE, width=value.....)

where, **x** is any R object (conceptually); typically numeric.

trim is logical; if FALSE, logical, numeric and complex values are right-justified to a common width: if TRUE, the leading blanks for are suppressed.

digits : It is used to state the number of significant digits to be used for numeric and complex x. The default, NULL, uses **getOption("digits")**.

Dealing with some useful functions–Contd.

nsmall: It is used to specify the minimum number of digits to the right of the decimal point in formatting real or complex numbers in non-scientific formats. **The permitted range is:0 <= nsmall <= 20**.
justify: It is used to mention whether a character vector should be right-justified, centred or left alone. Default , it is left-justified .
width: It is used to specify the minimum field width or NULL or 0 for no restriction.
na.encode: It is logical implying whether the NA strings be encoded or not. It may be noted here that numerical, complex nor logical Nas are always encoded as "NA".
scientific: It is used to specify logically whether the elements of a real or complex vector should be encoded in scientific format, or an integer penalty is to given. If nothing is mentioned, then the values correspond to the current default penalty.
... Used to imply that one or more arguments can be specified.

Illustrative Examples with format()

>x=1234.56789321
>format(x)
[1] "1234.568" # This is shown with default number of digits including the digits after the decimal point is 7,hence only 3 digits come after the decimal point
>format(x,digits=10)
[1] "1234.567893" # Here, a total of 10 digits are displayed
>format(x,digits=2)
[1] "1235" # Here maximum number of significant digits mentioned is 2, as it has four before the decimal point, it shows all those digits after rounding the digits after the decimal point
>format(x,digits=6)
[1] "1234.57" # Here maximum number of significant digits mentioned is 6,so it shows two digits after the decimal point with rounded form including the four before the decimal point.
>format(x,digits=6,nsmall=3)
[1] "1234.568" # Here, in spite of the mentioning of 6 as the maximum number of digits to be shown, it is overridden by nsmall=3 and hence the minimum number of digits after the decimal point must be 3 that totals to 7 digits.
>format(x,digits=6,nsmall=2)
[1] "1234.57" **>format(x,digits=6,nsmall=2,scientific=TRUE)**
[1] "1.23457e+03" # Here,nsmall=2 is maintained even with the scientific notation.
>format(c(6.22, 13.123), digits = 5, nsmall = 1)
[1] " 6.220" "13.123" # nsmall states the minimum number of digits after the decimal point.For a vector,size of all the numbers must be same. Hence, it shows 6.220 to match with 13.123.
The following example also explains this:
>format(c(6.22, 13.123), digits = 4, nsmall = 1)
[1] " 6.22" "13.12"
By default, **format() pads the strings with spaces** so that they all have the same length.
>format(c("A", "AB", "ABC"), width = 6, justify = "centre")
[1] " A " " AB " " ABC "
>format(c("A", "AB", "ABC"), width = 6, justify = "centre",trim=TRUE)
[1] " A " " AB " " ABC "
>format(c("A", "AB", "ABC"), justify = "centre",trim=TRUE)
[1] " A " "AB " "ABC" # The same size of all strings of a vector overrides the **trim=TRUE** option.
>format(c("A", "AB", "ABC"), width = 6, justify = "left")
[1] "A " "AB " "ABC "

Illustrative Examples with format()

More examples:

```
>format(1:10, trim = TRUE)
 [1] "1" "2" "3" "4" "5" "6" "7" "8" "9" "10"
>df = data.frame("(row names)"= c("aaaaa", "b"), check.names = FALSE)
>df
(row names)
1                                aaaaa
2                                b
>format(df)
(row names)
1                                aaaaa
2                                b
>format(df, justify = "left")
(row names)
1 aaaaa
2 b
>z =list(a = letters[1:3], b = (-pi+0i)^((-2 : 2)/2), c = c(1, 10, 100, 1000),
+ d = c("a", "longer", "character", "string"),
+ q = quote( a + b ), e = expression(1+x))
>z
$a
[1] "a" "b" "c"
$b
[1] -0.3183099+0.0000000i 0.0000000-0.5641896i 1.0000000+0.0000000i
[4] 0.0000000+1.7724539i -3.1415927+0.0000000i
$c
[1] 1 10 100 1000
$d
[1] "a" "longer" "character" "string"
$q
a + b
$e
expression(1 + x)
```

More Examples of format()

```
>format(z, digits = 2)
                                                                    a
                                                  "a, b, c"
b
"-0.32+0.00i, 0.00-0.56i, 1.00+0.00i, 0.00+1.77i, -3.14+0.00i"
c
                                                  "1, 10, 100, 1000"
d
                          "a , longer , character, string "
                                                                    q
                                                  "a + b"
                                                                    e
                                                                    "expression(1 + x)"
>format(z, digits = 2, justify = "left", trim = FALSE)
                                                                    a
                                                  "a, b, c"
                                                                    b
"-0.32+0.00i, 0.00-0.56i, 1.00+0.00i, 0.00+1.77i, -3.14+0.00i"
                                                                    c
                                                  " 1, 10, 100, 1000"
                                                                    d
                          "a , longer , character, string "
                                                                    q
                                                  "a + b"
                                                                    e
                                                                    "expression(1 + x)"
>format(c("A", "AB", "ABC"), width = 6, justify = "right",trim=TRUE)
" A" " AB" " ABC" # The width option overrides the trim=TRUE option
>format(c("A", "AB", "ABC"), width = 6, justify = "none")
[1] "A" "AB" "ABC" # none option overrides the 'same size' criteria. Hence all spaces are removed.
```

For printing large quantities with a sequenced format we can use the arguments big.mark or big.interval. For instance, here is how we can print a number with sequences separated by a comma ","

```
>format(123456789, big.mark = ",")
[1] "123,456,789"
```

The apply family in R

R has many apply functions which are to be very useful to simplify our code. The apply functions are all covered in dplyr package but it is still good to know the differences and how to use it. It is just too convenient to ignore them.

Let us first have a birds' eye view of the mnemonics of various term of the family that we are going to study.

Mnemonics

- **apply** is generic: applies a function to a matrix's rows or columns (or, more generally, to dimensions of an array)
- **lapply** is a list apply which acts on a list or vector and returns a list.
- **sapply** is a simple lapply(function defaults to returning a vector or matrix when possible)
- **vapply** is a verified apply (allows the return object type to be prespecified)
- **rapply** is a recursive apply for nested lists, i.e. lists within lists
- **tapply** is a tagged apply where the tags identify the subsets

The syntax of the functions are stated below:

The apply family in R–Syntax

apply(x,margin,fun, . . .), where, **x** is an array including a matrix; margin is a vector giving the subscripts which the function will be applied over. E.g., for a matrix 1 indicates rows, 2 indicates columns, c(1,2) indicates rows and columns. Where x has named dimnames, it can be a character vector selecting dimension names and **fun** is the function to be applied. In the case of functions like **+,%*%**, etc., the function name must be backquoted or quoted. The **ellipsis(. . .)** implies the optional arguments to fun
Let us be more elaborate.

- If **x** is not an array but an object of a class with a non-null value (such as a data frame), then the apply attempts to coerce it to an array via as.matrix if it is two-dimensional (e.g., a data frame) or via as array..array.
- **FUN** calls match.fun to find out the match and it is either a function or a symbol (e.g., a backquoted name) or a character string pointing out a function to be searched for from the environment of the call to apply.
- Arguments in ...must have different names as already stated ones. We must be careful enough to avoid any partial matching to MARGIN or FUN. In general,we should mention the first three arguments if . . . is passed.

The apply family in R–Syntax–Contd.

If each call to **FUN** returns a vector of length n, then apply returns an array of dimension **c(n,dim(X)[MARGIN]**,if n>1 . If n equals 1, apply returns a vector if MARGIN has length 1. If n is 0, the result has length 0 but not necessarily the **'correct'** dimension.

If the **calls to FUN** return vectors of different lengths, apply returns a list of length **prod(dim(x)[MARGIN])** with dim set to **MARGIN**. if this has length greater than one.

In all cases the result is coerced by **as.vector** to one of the basic vector types before the dimensions are set, so that (for example) factor results will be coerced to a character array.

lapply(x, FUN,. . . .) #lapply() function does not need MARGIN.

sapply(x,FUN,. . . ,simplify=TRUE,USE.NAMES=TRUE)

vapply(x,FUN,FUN,VALUE,. . . ,USE.NAMES=TRUE)

Where,

Simplify: logical or character string; should the result be simplified to a vector, matrix or higher dimensional array if possible? For **sapply** it must be named and not abbreviated. The default value, TRUE, returns a vector or matrix if appropriate, whereas if **simplify = "array"** the result may be an array of **"rank" (=length(dim(.)))** one higher than the result of **FUN(X[[i]])**.

The apply family in R–Syntax–Contd.

USE.NAMES: logical; if TRUE and if X is character, use X as names for the result unless it had names already. Since this argument follows ... its name cannot be abbreviated.
FUN.VALUE: a (generalized) vector; a template for the return value from **FUN**.
n integer: the number of replications.
Expr: the expression (a language object, usually a call) to evaluate repeatedly.
a list, typically returned from **lapply()**.
logical; if true, simplify2array() will produce a ("higher rank") array when appropriate, whereas higher = FALSE would return a matrix (or vector) only. These two cases correspond to sapply(*, simplify = "array") or simplify = TRUE, respectively.
tapply(x, index, FUN=NULL)
Here, **index** is "a list of factors, each of same length as x". This is particularly applied where we may want to perform the apply function on some data, but have it separated by factor. Here, index is the factor we want to use to separate the data.

The apply family in R–Syntax–Contd.

rapply(x,FUN,class)

where,

- first argument in the rapply function is the list, here it is x.
- the second argument is the function that needs to be applied over the list.
- last argument gives the classes to which the function should be applied

Illustrative uses of the referenced functions

```
># Matrix for demonstration
>myMatrix=matrix(floor(runif(10,0,50)),4,5)
>myMatrix
[,1] [,2] [,3] [,4] [,5]
[1,] 46 15 42 27 3
[2,] 0 0 34 14 20
[3,] 27 3 46 15 42
[4,] 14 20 0 0 34
```

Illustrative uses of the referenced functions–Contd.

```
># Let us now apply min to rows of the matrix to obtain the minimum value from each of the rows
>apply(myMatrix,1,min)
[1] 3 0 3 0
>apply(myMatrix,1,mean)
[1] 26.6 13.6 26.6 13.6
>l=list(2,4,6,c(5,6,7))
>lapply(l,FUN=mean) # Applying the mean function over the list ' l'
[[1]]
[1] 2
[[2]]
[1] 4
[[3]]
[1] 6
[[4]]
[1] 6
>L=list(c(2,4,6),c(8,6,12,16))
```

Illustrative uses of the referenced functions–Contd.

>**lapply(L,mean)** # Applying the mean function over the list ' L' to calculate mean for each of the vectors.

[[1]]
[1] 4
[[2]]
[1] 10.5

>**lapply(L,min))** # Applying the mean function over the list ' L' to calculate min for each of the vectors to the minimum value from each of the vectors

[[1]]
[1] 2
[[2]]
[1] 6

>**lapply(L,sum))** # Applying the sum function over the list ' L' to calculate sum for each of the vectors to obtain the total of each of the vectors

[[1]]
[1] 12
[[2]]
[1] 42

Illustrative uses of the referenced functions–Contd.

lst=list(a=4:8,b=5:10)
>sapply(lst,mean)
a b
6.0 7.5
lst=list(a=4:8,b=5:10)
vapply(lst,mean,numeric(1)) # The third argument means single numeric value is required.
a b
6.0 7.5
vapply(lst,mean,1i) # here,1i implies single complex value(imaginary)
a b
6.0+0i 7.5+0i
So, we observe that **vapply** needs a third argument : **FUN.VALUE** where we specify the type of data we expect.
Attending_Doctors=data.frame(Doctor_Name=c("Dr. S. Maity","Dr. R. Palaudhi","Dr.C. Roy","Dr. A. Ghosh"),Gender=c("Male","Female","Female","Male"), Fees=c(500,300,700,400))

Illustrative uses of the referenced functions–Contd.

```
>Attending_Doctors
 Doctor_Name Gender Fees
1 Dr. S. Maity Male 500
2 Dr. R. Palaudhi Female 300
3 Dr.C. Roy Female 700
4 Dr. A. Ghosh Male 400

tapply(Attending_Doctors$Fees,Attending_Doctors$Gender,mean)
Female    Male
 500       450
tapply(Attending_Doctors$Fees,Attending_Doctors$Gender,min)
Female    Male
 300       400
```

Here, **tapply()** has been demonstrated in the code above.It applies a function or operation on a subset of a vector broken down by a given factor variable..In the above figure, we have defined a dataframe named **Attending_Doctors**. Now from the data frame , we tried to find out the average fees of the doctors grouped by the factor variable **Gender**.

Illustrative uses of the referenced functions–Contd.

We have also shown minimum fees under each category,by using the **min** function. Now, we shall observe the application of **rapply( )**. This function is meant for recursive apply. It is particularly useful for a list structure which is nested. In particular, it is used to apply a function to all elements of a list recursively. Let us have a look on its application below:

```
lst=list(2,3,4,5,5)
>rapply(lst,function(lst)sqrt(lst),class=c("numeric"))
[1] 1.414214 1.732051 2.000000 2.236068 2.236068
>rapply(lst,function(lst)sqrt(lst),how="replace")
[[1]]
[1] 1.414214
[[2]]
[1] 1.732051
[[3]]
[1] 2
[[4]]
[1] 2.236068
```

String Concatenation in R

In R string concatenation is done by using the paste function. The syntax of the function is:

paste(..., sep = " ", collapse = NULL), where the **ellipsis(.....)** is replaced with one or data values that may be either strings or other values which can be coerced to characters. **'sep'** is to specify to separate the terms and **'collapse'** contains an optional character string to separate the results.

Example-1. paste("Today's date is:",date())

[1] "Today's date is: Fri Aug 03 16:21:04 2018"

Example-2:

```
>str1="Ram"
>str2="ayan"
>paste(str1,str2)
[1] "Ram ayan"
>paste(str1,str2, sep="")
[1] "Ramayan"
>paste(str1,str2, sep="", collapse=NULL)
[1] "Ramayan"
```

String Concatenation in R–Contd.

Example-3.
>s1="Emp"
>s2="Id"
paste(s1,s2,sep='-')
[1] "Emp-Id"
We do hope that the above examples are enough to understand concatenation.
Forming Row and Column Sums and Means
This types calculation is required for numeric arrays,matrices (or data frames).
The following functions are available for the purpose:
colSums (x, na.rm = FALSE, dims = 1)
rowSums (x, na.rm = FALSE, dims = 1)
colMeans(x, na.rm = FALSE, dims = 1)
rowMeans(x, na.rm = FALSE, dims = 1)
.colSums(x, m, n, na.rm = FALSE)
.rowSums(x, m, n, na.rm = FALSE)
.colMeans(x, m, n, na.rm = FALSE)
.rowMeans(x, m, n, na.rm = FALSE)
Arguments:

Row and Column Sums and Means–Contd.

x An array of two or more dimensions, containing, numeric, complex, integer or logical values or a numeric data frame. For **colSums()** etc., a numeric, integer or logical matrix(or vector of length m*n)
Na.rm Logical. Should missing values including NanN be omitted from the calculations. **dims** Integer: Which dimensions are regarded as 'rows' or 'columns' to sum over. For row*, the sum or mean is over dimensions dims+1,. . . .;for col*,it is over dimensions 1:dims
m, n The dimensions of the matrix x for **.columns() etc.**

C-style Formatting of R data.
R comes with the **sprintf()** function used in c language for the formatting of strings. Precisely speaking,it is a wrapper function The syntax of the function is as shown below:
. **sprintf("format string with or without text",values/variable to be printed as per the formatting symbol)**

C-style Formatting of R data–Contd.

Notation	Description
%s	A string
%d,%i	An integer
%0xd	An integer padded with x leading zeroes
%f	Decimal notation with 6 digits after the decimal point
%.xf	Floating point number with x digits after the decimal point
%e	Compact scientific notation with e in the exponent
%E	Compact scientific notation with E in the Exponent
%g	Compact decimal or scientific notation with e in the exponent

Syntax of the Format Slot

The full syntax for a format slot is defined by:

%[parameter][flags][width][.precision][length]type

The percent symbol, **%** indicates a placeholder or slot.

The parameter field is an optional field that can take the value n$ in which n is the number of the variable to display, allowing the variables provided to be used multiple times, using varying format specifiers or in different orders.

The flags field can be zero or more (in any order) of:

- - **(minus)** Left-align the output of this placeholder.
- + (plus) Prepends a plus for positive signed-numeric types.
- ' ' (space) Prepends a space for positive signed-numeric types.
- 0 (zero) When the 'width' option is specified, prepends zeros for numeric types.
- # (hash) Alternate form:
- for g and G types, trailing zeros are not removed.
- for f, F, e, E, g, G types, the output always contain a decimal point.
- for o, x, X types, the text 0, 0x, 0X, respectively, is prepended to non-zero numbers.

Syntax of the Format Slot–Contd.

The width field is an optional field that we use to specify a minimum number of characters to output, and is typically used to pad fixed-width fields in tabulated output, where the fields would otherwise be smaller, although it does not cause truncation of oversized fields.
For example, the command:
sprintf("%*d", 10, 25) will generate the following output:
[1] " 25" . with 8 leading spaces.
The precision field usually specifies a maximum limit on the output, depending on the particular formatting type.For example,
>**sprintf("%.*s", 4, "abcdefgh")**
"abcd"
The length field is also optional.
The most important field is the **type** field. The following points may be kept in mind:

Formatting–Contd.

- **%:** Prints a literal % character (this type doesn't accept any flags, width, precision, length fields).
- **d, i:** integer value as signed decimal number.
- **f:** double value in normal fixed-point notation.
- **e, E:** double value in standard form.
- **g, G:** double value in either normal or exponential notation.
- **x, X:** unsigned integer as a hexadecimal number. x uses lower case, while X uses upper case.
- **o:** unsigned integer in octal notation.
- **s:** null terminated string.
- **a, A:** double value in hexadecimal notation

Example: File Names

While working on data analysis projects, it is common to generate different files with similar names (e.g. either for creating images, or data files, or documents). Let us suppose that we need to generate the names of 5 data files (with **.csv** extension).

Formatting–Contd.

All the files have the same prefix name but each of them has a different number: data01.csv, data02.csv, and data03.csv,data04.csv,data05.csv.

One simple solution to this problem is to generate a character vector with these names in R with the following coding:

file_names = c('data01.csv', 'data02.csv', 'data03.csv','data04.csv', 'data05.csv')

This is a very tedious process; as we may need to handle a large number of files together. So, we find out a better solution to handle this situation in a more efficient way with the **past0() function** as stated below:

file_names =paste0('data0', 1:5, '.csv')

But there is limitation to this option also. In case, we need to generate 999 such file names, we shall have to code like the following:

files1 = paste0('data00', 1:9, '.csv')
files2 =paste0('data0', 10:99, '.csv')
files3=paste0('data',100:999,'.csv')
file_names=c(files1, files2,file3)

Still there is a better way of doing this by using **sprintf()** with the **%0xd option** to indicate that an integer should be padded with x leading zeros. For instance, the 999 file names can be generated by writing only the following command:

file_names = sprintf('data%002d.csv', 1:999)

Formatting–Contd.

In case of the preceding command,the first nine elements in file_names will include two leading zeroes, next 99 file names with one leading zero and the remaining ones without any leading zero.

More Examples:

We should be careful with the format of **sprintf()**: most things in R are floats only integer-valued reals get coerced to integer.

sprintf("%s is %f feet tall", "Mr.Singh", 6.5) # This is correct and we shall get the following output:

"Mr.Singh is 6.500000 feet tall"

But if we write, **try(sprintf("("%s is %f feet tall", "Mr.Singh", 6.5))**

It will generate the following error:

Error: unexpected symbol in "try(sprintf("("%s is %f feet" This is because %i is used for integer values.

Let us now try the following:

try(sprintf("%s is %f feet tall", "Mr.Singh", 6.5))

The command runs successfully and produces the result:

[1] "Mr.Singh is 6.500000 feet tall"

Formatting–Contd.

Let us now check the use of a literal % :

sprintf("%.2f%% said 'yes' (out of a sample of size %.0f)", 87.654, 100)

When the above command is run, it will generate the following output: **[1] "87.65% said 'yes' (out of a sample of size 100)"**

Let us now check the output of the various formats of **pi** :

```
>sprintf("%f", pi)
[1] "3.141593"
>sprintf("%.2f", pi)
[1] "3.14"
>sprintf("%1.0f", pi)
[1] "3"
>sprintf("%5.2f", pi)
[1] " 3.14"
>sprintf("%05.2f", pi)
[1] "03.14"
>sprintf("%+f", pi)
[1] "+3.141593"
>sprintf("% f", pi)
[1] " 3.141593"
>sprintf("%-10f", pi) # left justified
[1] "3.141593 "
>sprintf("%e", pi)
[1] "3.141593e+00"
>sprintf("%E", pi)
[1] "3.141593E+00"
>sprintf("%g", pi)
[1] "3.14159"
```

Formatting–Contd.

```
>sprintf("%g", 1e6 * pi)  ->exponential
[1] "3.14159e+06"
>sprintf("%.9g", 1e6 * pi) # ->"fixed"
[1] "3141592.65"
>sprintf("%G", 1e-6 * pi)
[1] "3.14159E-06"
>## no truncation:
>sprintf("%1.f",101)
[1] "101"
```

Let us now re-use one argument three times to see the difference between %x and %X

```
x = sprintf("%1$d %1$x %1$X", 0:20)
```

If we print the output, we shall see the following:

```
>x
 [1] "0 0 0" "1 1 1" "2 2 2" "3 3 3" "4 4 4" "5 5 5"
 [7] "6 6 6" "7 7 7" "8 8 8" "9 9 9" "10 a A" "11 b B"
[13] "12 c C" "13 d D" "14 e E" "15 f F" "16 10 10" "17 11 11"
[19] "18 12 12" "19 13 13" "20 14 14"
```

To get the output without the enclosing quotation marks,, we may write:

```
noquote(format(x, justify="right"))
```

Its output will be :

```
[1] 0 0 0 1 1 1 2 2 2 3 3 3 4 4 4 5 5 5 6 6 6 7 7 7
 [9] 8 8 8 9 9 9 10 a A 11 b B 12 c C 13 d D 14 e E 15 f F
[17] 16 10 10 17 11 11 18 12 12 19 13 13 20 14 14
```

A More sophisticated use is shown below:

```
sprintf("min 15-char string '%15s'",
 c("a", "ABC", "and any longer string you like"))
```

The output of the above command is shown below:

```
[]1] "min 15-char string ' a'"
[2] "min 15-char string ' ABC'"
```

Let us now have look at the following code:

```
n =1:20
sprintf(paste("e with %2d digits = %.",n,"g",sep=""), n, exp(1))
```

The output of the above commands will be as shown in the slide below:

Formatting–Contd.

```
 [1] "e with 1 digits = 3"
 [2] "e with 2 digits = 2.7"
 [3] "e with 3 digits = 2.72"
 [4] "e with 4 digits = 2.718"
 [5] "e with 5 digits = 2.7183"
 [6] "e with 6 digits = 2.71828"
 [7] "e with 7 digits = 2.718282"
 [8] "e with 8 digits = 2.7182818"
 [9] "e with 9 digits = 2.71828183"
[10] "e with 10 digits = 2.718281828"
[11] "e with 11 digits = 2.7182818285"
[12] "e with 12 digits = 2.71828182846"
[13] "e with 13 digits = 2.718281828459"
[14] "e with 14 digits = 2.718281828459"
[15] "e with 15 digits = 2.71828182845905"
[16] "e with 16 digits = 2.718281828459045"
[17] "e with 17 digits = 2.7182818284590451"
[18] "e with 18 digits = 2.71828182845904509"
[19] "e with 19 digits = 2.718281828459045091"
[20] "e with 20 digits = 2.7182818284590450908
```

sprintf() is a very powerful function. If we use formatting symbols even out of normal order, it will print the values correctly by rearranging them depending on the mentioned values as shown through following command:

sprintf("Second value is:%2$1.0f, first value is: %1$5.2f, and the third value is: %3$1.0f", exp(1), 2, 3)

The output of the above command will be:

[1] "Second value is:2, first value is: 2.72, and the third value is: 3"

Formatting–Contd.

We can also use asterisk for width or precision as illustrated below:

sprintf("precision %.*f, width '%*.5f'", 3, pi, 10, pi)

The output of the above will be:

[1] "precision 3.142, width ' 3.14159'"

Re-use of asterisk and argument: **'e' example repeated:**

sprintf("e with %1$2d digits = %2$.*1$g", n, exp(1))

The output of the above command is shown below:

```
[1] "e with 1 digits = 3"
 [2] "e with 2 digits = 2.7"
 [3] "e with 3 digits = 2.72"
 [4] "e with 4 digits = 2.718"
 [5] "e with 5 digits = 2.7183"
 [6] "e with 6 digits = 2.71828"
 [7] "e with 7 digits = 2.718282"
 [8] "e with 8 digits = 2.7182818"
 [9] "e with 9 digits = 2.71828183"
```

The arguments can also be re-cycled as shown below:

sprintf("%s %d", "test", 1:3)

The output will be as shown below:

[1] "test 1" "test 2" "test 3"

Dates and Times in R

R provides several options for dealing with date and date/time data. The built-in **as.Date() function** handles dates (without times); the contributed library chron handles dates and times, but does not control for time zones; and the **POSIXct and POSIXlt** classes allow for dates and times with control for time zones. The general rule for date/time data in R is to use the simplest technique possible. Thus, for date only data, **as.Date()** will usually be the best choice. If we need to handle dates and times, without timezone information, the chron library is a good choice; the POSIX classes are especially useful when timezone manipulation is important. dates are stored internally as the number of days or seconds from some reference date. Thus dates in R will generally have a numeric mode, and the class function can be used to find the way they are actually being stored. The **POSIXlt class** stores date/time values as a list of components (hour,min, sec, mon, etc.) making it easy to extract these parts.
To get the current date, the **Sys.Date()** function will return a Date object which can be converted to a different class if necessary.

Dates and Times in R–Contd.

The **as.Date** function allows a variety of input formats through the format= argument. The default format is a four digit year, followed by a month, then a day, separated by either dashes or slashes. Some examples of dates are shown below which as.Date will accept by default:

```
>d=as.Date('1960/08/24')
>d
[1] "1960-08-24"
>md=as.Date('1986-07-21')
>md
[1] "1986-07-21"
```

But, if the given date is not in the default format of 'yyyy/mm/dd' or 'yyyy-mm-dd', then we are required to specify the format for storing it as a date. The format option is required for this purpose as illustrated below:

```
>sd=as.Date('01-05-1987',format='%d-%m-%Y')
>sd
[1] "1987-05-01"
```

The following table shows the symbols that can be used in a format option.

Dates and Times in R–Contd.

Code	Value
%d	Day of the month (decimal number)
%m	Month (decimal number)
%b	Month (abbreviated)
%B	Month (full name)
%y	Year (2 digit)
%Y	Year (4 digit)

Internally, Date objects are stored as the number of days since January 1, 1970, using negative numbers for earlier dates. The **as.numeric()** function can be used to convert a Date object to its internal form. For example, we take the date stored in md earlier to convert the into a number as illustrated below: >**mdd=as.numeric(md)**

>**mdd**

[1] 6045

To extract the components of the dates, the weekdays, months, days or quarters functions can be used. For example, to find the day of the week on which someone with a given date was born, we proceed as follows:

Dates and Times in R–Contd.

```
>bday=c(mine=as.Date("1958-07-04"))
>weekdays(bday)
 mine
"Friday"
```

calculations with dates:

To find the difference between dates:

```
dt1 - dt2
## Time difference of 459 days
difftime(dt1, dt2, units = "weeks")
Time difference of 65.57 weeks
```

Add or subtract days:

```
dt2 + 10
## [1] "2011-04-30"
dt2 - 10
## [1] "2011-04-10"
```

create a vector of dates and find the intervals between them:

```
three.dates = as.Date(c("2010-07-22", "2011-04-20", "2012-10-06"))
```

Dates and Times in R–Contd.

```
three.dates
## [1] "2010-07-22" "2011-04-20" "2012-10-06"
diff(three.dates)
## Time differences in days
## [1] 272 535
```

create a sequence of dates:

```
six.weeks = seq(dt1, length = 6, by = "week")
six.weeks
## [1] "2012-07-22" "2012-07-29" "2012-08-05" "2012-08-12" "2012
08-19"
## [6] "2012-08-26"
six.weeks = seq(dt1, length = 6, by = 14)
six.weeks
## [1] "2012-07-22" "2012-08-05" "2012-08-19" "2012-09-02" "2012
09-16"
## [6] "2012-09-30"
six.weeks = seq(dt1, length = 6, by = "2 weeks")
```

Dates and Times in R–Contd.

six.weeks

[1] "2012-07-22" "2012-08-05" "2012-08-19" "2012-09-02" "2012 09-16"

[6] "2012-09-30"

The **chron()** function creates chronological objects representing dates and times of day. Its syntax is as shown below:

chron(dates.,times., format=c(dates="m/d/y",times="h:m:s"), out.format,origin.)

where, **dates.** contain character or numeric vector specifying dates. If character, dates. are assumed to be in one of the date formats below; if numeric, dates. are assumed to be Julian dates, i.e., number of days since origin..

times. contain optional character or numeric vector specifying times of day. If character, times. are assumed to be in one of the time formats below; if numeric, times. are assumed to be fractions of a day.

format is a vector or list specifying the input format of the input.

Dates and Times in R–Contd.

The format can be either strings specifying one of the recognized formats below or a list of user-supplied functions to convert dates from character into **Julian dates** and vice versa. The dates format can be any permutation of the characters "d", "m", or "y" delimited by a separator (possibly null), e.g.,"m/d/y", "d-m-y", "ymd", are all valid; the format can also be permutations of the words "day", "month" and "year"(with non-null separator), which produces the month name, e.g., "month day year" produces **"April 20 1992"**, **"day mon year"** produces **"20 Apr 1992"**. The t**imes format** can be any permutation of "h", "m", and "s" separated by any one non-special character. The default is "h:m:s".

out.format is a vector or list specifying date and time format for printing and output. Default is same as format.

origin. is a vector specifying the date with respect to which Julian dates are computed. Default is c(month = 1, day = 1, year = 1970); we may set the option **chron.origin** to specify our own default, e.g.,**options(chron.origin = c(month=1, day=1, year=1990)**

Dates and Times in R–Contd.

The dates and times are provided to the **chron** function as separate values, so some preprocessing may be necessary to prepare input date/times for the chron function. When using character values, the default format for dates is the decimal month value followed by the decimal day value followed by the year, using the slash as a separator. Alternative formats can be provided by using the codes shown in the Table below:

Format codes for dates:

Code	Value
m	Month (decimal number)
d	Day of the month (decimal number)
y	Year (4 digit)
mon	Month (abbreviated)
month	Month (full name)

Format codes for times:

Code	Value
h	Hour
m	Minute
s	Second

Dates and Times in R–Contd.

Let us now study the functions available with **Time** .

Sys.time() and **now()** return the **date, time and the timezone**. In **now()**, we can specify the timezone using the **tzone** argument. We observe the following:

>**Sys.time()** returns the following:

[1] "2021-12-07 22:02:46 IST"

Lubridate package in R is used to store and modify the date formats. It has a large variety of parse functions available, which allow the access of dates and various formats with great simplicity. **dmy_hms ()** method in this package is the most common method, used to store the dates in the standard format **(data-month-year-hour-minute-second)**.

To see the today's date using the lubridate package, we write:

>**lubridate::today()**

We get a response like the following one:

[1] "2021-12-07"

Now, if we issue the following command:

>**lubridate::now()**

Dates and Times in R–Contd.

This shows us the current date in the following IST format:
[1] "2021-12-07 22:04:27 IST"
UTC, or Universal Time Coordinated, is the most precise and commonly referred to time standard. Since the 1970s, this time standard has been globally used as the most precise time standard, instead of formerly used GMT standard, which has turned now into a regular time zone. In order to know the current time in UTC, we can specify the **tzone parameter as 'UTC'** as illustrated below.
>**lubridate::now(tzone="UTC")**
[1] "2021-12-07 16:50:42 UTC"
The **am() and pm()** functions allow us to check whether date/time occur in the AM or PM. **They return a logical value i.e. TRUE or FALSE** as shown below:
>**x=lubridate::now()**
>**lubridate::am(x)**
[1] FALSE
>**lubridate::pm(lubridate::now())**
[1] TRUE

Dates and Times in R–Contd.

To check whether the year in the today's date is a leap_year or not, we can write as under:

>**lubridate::leap_year(lubridate::today())**

The response is a Boolean value **TRUE or FALSE** as shown below as the response of the preceding command.

[1] FALSE

Let us now recall the references to the **POSIXct and POSIXlt** classes used in R.The **POSIXct class** stores date/time values as the number of seconds since January 1, 1970, while the **POSIXlt class stores them as a list** with elements for second, minute, hour, day, month, and year, among others. Unless we need the list nature of the **POSIXlt class, the POSIXct class** is the usual choice for storing dates in R.

Let us store the current date and time in an R string variable as shown below:

>**current_date='08/12/2021 12:24:30'**

To convert the string date in current_date in the **POSIXct format**, we write:

>**posix_date=as.POSIXct(current_date, format = "%m/%d/%Y %H:%M:%S")**

This generates a timestamp based on the string date-time.We may like to print it as under:

>**posix_date**

Dates and Times in R–Contd.

[1] "2021-08-12 12:24:30 IST"

Now, to extract time from the timestamp, we may write:

>**current_time =format(posix_date,format="%H:%M:%S")**

>**current_time**

This returns the following:

[1] "12:24:30"

The two most important functions in this regard are **strptime**, for inputting dates, and **strftime**, for formatting dates for output. Both of these functions use a variety of formatting codes to specify the way dates are read or printed. For example, dates in many logfiles are printed in a format like **"08/Dec/2021:12:55:20"**. To create a POSIXct date from a date in this format, the following call to **strptime** could be used:

>**mydate = strptime('08/Dec/2021:12:55:25',format= '%d/%b/%Y:%H:%M:%S')**

>**mydate**

[1] "2021-12-08 12:55:25 IST"

The **strftime ()** is a built-in R function that converts the time to character objects.The format methods and strftime return character vectors representing the time.Its syntax is: **strftime(x, format = "", tz = "", usetz = FALSE, . . .)**

Dates and Times in R–Contd.

Let us now the class of the timestamp generated by using **as.POSIXct()**.We write:

>**class(posix_date)**

This shows the following:

[1] "POSIXct" "POSIXt"

POSIXt is the parent class of both **POSIXct** and **POSIXlt**. This means that the classes both **POSIXct** and **POSIXlt** have been inherited from **POSIXt** for representing both date and time.

Let us now use **strftime()** over the timestamp as follows:

>**strftime=strftime(posix_date)**

Now,we check the class of this converted timestamp as shown below:

>**class(strftime)**

This shows the following as expected.

[1] "character"

When using POSIX dates, the optional **usetz=TRUE** argument to the format function can be specified to indicate that the time zone should be displayed.

Dates and Times in R–Contd.

Additionally, **as.POSIXlt** and **as.POSIXct** can also accept **Date or chron objects**, so they can be input as described previously and converted as needed. Conversion between the two POSIX forms is also possible.

The individual components of a **POSIX date/time object** can be extracted by first converting to **POSIXlt** if necessary, and then accessing the components directly as illustrated below:

```
>mydate = as.POSIXlt('2021-12-08 13:48:25')
>mydate$mday
[1] 8
>mydate$mon
[1] 11
$year
[1] 121
>mydate$hour
[1] 13
>mydate$sec
[1] 25
>mydate$min
[1] 48
```

Times in R–Contd.

The default input format for **POSIX dates** consists of the year, followed by the month and day, separated by slashes or dashes; for date/time values, the date may be followed by white space and a time in the form hour:minutes:seconds or hour:minutes; thus, the following are examples of valid **POSIX date** or date/time inputs:

1915/6/16
2005-06-24 11:25
1990/2/17 12:20:05

If the input times correspond to one of these formats, **as.POSIXct** can be called directly:

>dts = c("2005-10-21 18:47:22","2005-12-24 16:39:58", "2005-10-28 07:30:05 PDT")
as.POSIXlt(dts)
[1] "2005-10-21 18:47:22" "2005-12-24 16:39:58"
[3] "2005-10-28 07:30:05"

If the input date/times are stored as the number of seconds from January 1, 1970, we can create **POSIX date** values by assigning the appropriate class directly to those values. Since most date manipulation functions refer to the **POSIXt psuedo-class**, be sure to include it as the first member of the class attribute.

Times in R–Contd.

```
dts = c(1127056501,1104295502,1129233601,
1113547501,1119826801,1132519502,1125298801,1113289201)
mydates = dts
class(mydates) = c('POSIXt','POSIXct')
mydates
[1] "2005-09-18 08:15:01 PDT" "2004-12-28 20:45:02 PST"
[3] "2005-10-13 13:00:01 PDT" "2005-04-14 23:45:01 PDT"
[5] "2005-06-26 16:00:01 PDT" "2005-11-20 12:45:02 PST"
[7] "2005-08-29 00:00:01 PDT" "2005-04-12 00:00:01 PDT"
```

Conversions like this can be done more succinctly using the structure function:

```
>mydates = structure(dts,class=c('POSIXt','POSIXct'))
```

R Environment and Scope

With a view to write functions in a proper way and to avoid unusual errors, we need to know the concept of environment and scope in R.

R Programming Environment

R Environment can be thought of as a collection of objects (functions, variables etc.). It is created when we first fire up the R interpreter by running some R code. Any variable we define in the code, becomes part of the environment.**The top level environment available to us at the R command prompt is the global environment known as R_GlobalEnv**. The Global environment can be referred to as **.GlobalEnv** in R scripts as well. We can use the **ls() function** to show the variables and functions defined in the current environment. Moreover, we can use the **environment() function** to know the current environment.

For clarity,let us take some example as shown below.

```
>x=5
>y=7
>func=function(z)z=0
```

R Environment and Scope–Contd.

>**ls()**
[1] "x", "y","func"
This will be displayed if the above codes are run first in R.
>**environment()**
<**environment: R_GlobalEnv**>
>**.GlobalEnv**
<**environment: R_GlobalEnv**>
So, in the above example, we see that **x,y,func** are in the **R_GlobalEnv** environment.; but **the variable z in the function func() is not in the environment**. It is because when we define a function, a new environment is created inside the global environment.
In reality, an environment has a frame, which contains all the defined objects and a pointer to the parent environment. Hence, z is in the frame of the new environment created by the function func. Its parent environment being **R_GlobalEnv**, the frame points to it. In this way ,there may be **cascaded environment**.

Example: Cascading of environments

Let us consider the following definition of some function:

```
>func=function(func_x){
func2=function(func_y){ print("Inside func2()")
 print(environment())
print(ls()) }
 func(7)
print("We are now inside func()")
print(environment()) }
print(ls())
```

Now, if we run the function from the command prompt of R, we shall get some displays like the following ones:

```
>func(5)
[1] "Inside func2()"
<environment:0x00000000c3bcd9 >
[1] "func_y"
[1] "We are now inside func()"
<environment 0x000000000c2a375 >
"func_x","g"
>environment()
<environment: R_GlobalEnv >
```

R Environment and Scope–Contd.

```
>ls()
[1]"func"
```

Thus, we have observed different functions in different environments. With their respective objects within the defined frame.

Scope of R programming

Let us consider the following example:

Let us consider the following code snippet:

```
>outer_function=function(){
 x=5
inner_function=function(){
y=7}}
 >z=9
```

Now, we are in a position to explain the concepts of Global and Local variable.

Global variables

The variables that exist throughout the execution of a program are known as global variables. Such variables can be accessed and modified from any part of the program. However, The concept also depends on the perspective of a function. For instance, in the code snippet cited above, both x and z are global variables from the perspective of the **inner_function.()**. However,

R Environment and Scope–Contd.

from the perspective of the **outer_function()**, x is a local variable and only z is a global variable. The variable y remains completely invisible to the **outer_function()**.

Local variables

The variables that exist only within a certain part of a program like a function called local variables. The local variables are released as and when the function completes its execution. Thus ,in our code snippet illustrated above, y is local to the **inner_function()** and x is local to the **outer_function()**. If we change the value of a local variable, the change will only be local to the respective function and cannot be available from outside. This is true for all situations even if the name of a local variable matches with that of a global variable. This is illustrated below:

```
>x=10 # Defining a Global variable
>y=25  # # Defining another Global variable
>outer_function=function(){
+  x=5 # Defining a Local variable with the same name of a Global variable
+  cat("Value of Local x is:",x)
+  inner_function=function(){
+  y=7 ## Defining another Local variable with the same name of a Global variable
+ cat(" The value of Local y is:",y)
+  }
+ inner_function() }
```

R Environment and Scope–Contd.

```
>z=9
>outer_function()
[1] "Value of Local x is: 5"
[1] " The value of Local y is: 7"
>sprintf("The value of Global x is: %d ", x)
[1] "The value of Global x is: 10 "
>sprintf("The value of Global y is: %d",y)
[1] "The value of Global y is: 25"
```

Modifying Global variables from a function

Within the scope of a function, global variable can be read for use; but if we want to assign a new variable to a global variable, it is not done, rather a new local variable is created instead. It is observed in the preceding example also. To make a such an assignment, we need to use **the super assignment operator** <<- . Whenever we use such an operator, the system searches for the global variable in the parent environment of the function, if not found there, goes one step further to find out from the parent of the parent environment until it reaches the global environment. If it does not find it there then it creates the variable and assigns the desired value at the global level.

Accessing global variables

Global variables can be read but when we try to assign to it, a new local variable is created instead. This has been illustrated in the code given below:

R Environment and Scope–Contd.

```
>x=10
>y=25
>outer_function=function(){
+ x=5
+ a<<-6
+ cat("Value of Local x is:",x)
+ inner_function=function(){
+  y=7
+  b<<-9
+ print(paste(" The value of Local y is:",y))
+ }
+  inner_function()
+ }

>z=9
>outer_function()
[1] "Value of Local x is: 5"
[1] " The value of Local y is: 7"
>sprintf("The value of Global x is: %d ", x)
[1] "The value of Global x is: 10 "
>sprintf("The value of Global y is: %d",y)
[1] "The value of Global y is: 25"
>sprintf("The value of the newly created global variable within the outer_function() is: %d",a)
"The value of the newly created global variable within the outer_function() is: 6" >sprintf("The value of the newly created
global variable within the inner_function() is: %d",b)
[1] "The value of the newly created global variable within the inner_function() is: 9"
```

Lexical scope and function closures in R

Variable scope

Almost all programming languages allow the definition of variables which are labels or tags of the main memory locations of a computer representing or pointing to some value that may be defined and re-defined at run-time. In most modern programming languages, functions can define local variables which can be used in addition to any inputs (formal parameters) of the function – these are very significant for the development of modular, re-usable code components. In particular, they help to avoid unwanted name clashes in the global name-space. If a function refers to a variable which is neither a formal parameter nor a local variable, then a rule is needed to determine which (if any) variable with that label is in scope for the function, so that the program can understand which value to use.

Dynamic scope

Under dynamic scope, if an "unknown" variable is referred to in a function, the idea is to use the version of the variable that is in scope at the time that the function was called (and apply this rule recursively) – this is the scoping rule used by the S-PLUS implementation of the S language.

Lexical scope and function closures in R–Contd.

Dynamic scope was common among early dynamic programming languages – including early implementations of LISP (and is still used in Emacs LISP), as it was quite intuitive and natural to implement using a stack-based approach similar to the stack-based approach to passing variables in and out of subroutines commonly used by machine code and assembly programmers. Despite being intuitively appealing, at least initially, there are a number of problems with dynamic scope in practice. In particular, we can't really know by code inspection whether or not a given section of code will run in all situations without actually running the code, as we can't know whether all variable bindings will resolve accurately. This is an issue even for dynamic languages, but is particularly problematic for strongly typed compiled languages, as it becomes difficult for the compiler to figure out the types of all variables correctly and therefore generate the appropriate byte-code. It is also very difficult for a function to have associated state – to do this, one must somehow get state variables into global name-space where they then become vulnerable to masking and name clashes

Lexical scope

Under lexical scoping rules, if an **"unknown"** variable is referred to in a function, the idea is to use the version that is **"in scope"** in the enclosing piece of code (and apply this rule recursively) — this is the scoping rule used by R (as R is built on top of a Scheme interpreter, a **LISP** derivative which emphasises lexical scope). Variable bindings can be all resolved, checked and verified at compile-time – this is safer, and in many other ways better. Most modern languages adopt lexical scoping, including most functional languages, such as **LISP**s (including **LISP-STAT**). The book **LISP-STAT** was published over 25 years ago, so it just goes to authenticate that there is nothing new about these functional programming approaches.
Given that R has been lexically scoped and has supported function closures since day one, it is reasonable to wonder why this programming style is not used more widely in R code. This is the difference in scoping rules between S-PLUS and R that has led to a fear of developing any R code which relies on non-local scoping rules.**Lexical scoping is very powerful, and exploited widely in functional programming styles.** The use of lexical scope and function closures in R is described quite nicely in Gentleman (2008), along with many other things.

Lexical scope–Contd.

To make sure that the concepts are clear, let us inspect the following piece of code and figure out what the result of the final function call will be.

```
>x=1
>y=2
>func1=function(x)
+ {
+  x*a + y
+ }
>func2=function(x)
+ {
+ x=2
+ y=1
+ func1(x)
+ }
>func2(2)
[1] 14
>x=5
>y=7
>func1=function(x)
+ {
+ x*a + y
+ }
>func2=function(x)
+ {
+ x=3
+ y=4
+  func1(x)
+ }
>func2(3)
[1] 25
```

Function closures

Some languages (including **LISP**s and derivatives such as Scheme, Python, and R) have functions as **"first class objects"**, which means that a function is able to return as its value another function. If the function (fChild) returned by the function (fParent) refers to variables not local to fChild, then scoping rules must apply to the resolution of the variable binding. If the language is lexically scoped, then the binding is determined by the variables in scope within the function fParent. The function fChild therefore has an **associated environment**, as we have seen earlier,which provides bindings for non-local variable references – this allows maintaining of state. A function together with its environment is called a **function closure**, and is a very powerful programming tool.

Function closures for scientific computing

Function closures have numerous important applications in a variety of problems in scientific computing that involve dealing in some way with the **"function environment problem"**. There is quite a nice discussion of this issue in Oliveira and Stewart (2006).

Useful functions in R

- **str(df name)** # Shows the structure of a data frame
  Example:
  str(Attending_Doctors)
  'data.frame': 4 obs. of 3 variables:
  $ Doctor_Name: Factor w/ 4 levels "Dr. A. Ghosh",..: 3 2 4 1
  $ Gender : Factor w/ 2 levels "Female","Male": 2 1 1 2
  $ Fees : num 500 300 700 400

- **class(df name)** # shows the object class of a data frame Example:
  class(Attending_Doctors)
  [1] "data.frame"

- **head(data frame name,n=2)** # Shows the first two rows from the data frame. Example:
  head(Attending_Doctors,n=3)
  Doctor_Name Gender Fees
  1 Dr. S. Maity Male 500
  2 Dr. R. Palaudhi Female 300
  3 Dr.C. Roy Female 700

Useful functions in R–Contd.

- **tail(data frame name,n=2)** # Shows the last two rows from the data frame
  Example:
  tail(Attending_Doctors,n=2)

	Doctor_Name	Gender	Fees
3	Dr.C. Roy	Female	700
4	Dr. A. Ghosh	Male	400

- **ls()** # lists objects in the current environment.
- **grep()** # Used for pattern matching.
  Syntax:
  grep(pattern, x, ignore.case = FALSE, perl = FALSE, value = FALSE, fixed = FALSE, useBytes = FALSE, invert = FALSE)
  where, **pattern :** character string containing a pattern that describes a set of strings (or character string for fixed = TRUE) to be matched in the given character vector. Coerced by **as.character** to a character string if possible. If a character vector of length 2 or more is supplied, the first element is used with a warning.

Useful functions in R–Contd.

x is a character vector where matches are sought, or an object which can be coerced by as.character to a character vector. Long vectors are supported
ignore.case: if FALSE, the pattern matching is case sensitive and if TRUE, case is ignored during matching.
value ,if **FALSE**, a vector containing the (integer) indices of the matches determined by grep is returned, and if TRUE, a vector containing the matching elements themselves is returned
fixed :logical If TRUE, pattern is a string to be matched as is. Overrides all conflicting arguments.
useBytes: logical. If TRUE the matching is done byte-by-byte rather than character-by-character.
invert: logical. If TRUE return indices or values for elements that do not match.

- **rep(x,n)** # Used for the replication of the vector x, n times Example:
  >**rep('*',30)**
  [1] "*"
  [19] "*" "*" "*" "*" "*" "*" "*" "*" "*" "*" "*" "*"

Useful functions in R–Contd.

>cat(rep('*',30))

*** ***

It may be noted that **cat()** displays replicated character without the enclosing quotation marks.

- **rev(x)** # lists the elements in x in reversed order
  Example: >v=c("computer science","computer applications","biological science","phylosophy")
  >rev(v)
  [1] "phylosophy" "biological science" "computer applications"
  [4] "computer science"
- **sub() ,gsub()** # Replacement functions which replaces the occurrence of a substring with some other specified substring **sub()** replaces the first occurrence of a substring. Its syntax is:
  sub(old,new,string)
  The **'old'** substring is searched in the **'string'**, if found, the first occurrence of **'old'** is replaced with **'new'** substring. **gsub()** replaces all the instances of a substring. Its syntax is: **gsub(old,new,string)**

Useful functions in R–Contd.

Examples:

>mywords="My India is Great since its birth.However, we have got freedom in 1947 only. The British ruled us for about 200 years"

>sub("My","Our",mywords)

[1] "Our India is Great since its birth.However, we have got freedom in 1947 only. The British ruled us for about 200 years"

>myword="Everybody should tell the truth;telling truth costs nothing"

>gsub("tell","say",myword)

#gsub can be used with regular expressions also as illustrated below:

>facts="There is only 1(one)moon in our earth;but the Saturn has 9(nine)such moon"

>gsub("[0-9]*"," ",facts)

[1] " T h e r e i s o n l y (o n e) m o o n i n o u r e a r t h ; b u t t h e s a t u r n h a s (n i n e) s u c h m o o n "

It may be observed that **gsub** with a regular expression has replaced here all occurrences of digits and inserted space in between all the characters. Again , The following may noted to see the difference.

>gsub("[0-9]*","",facts)

[1] "There is only (one)moon in our earth;but the Saturn has (nine)such moon"

Useful functions in R–Contd.

- **unique(x)** # Removes duplicate elements from the vector x
  >**y=c(2,3,2,5,4)**
  >**unique(y)**
  [1] 2 3 5 4
- **system("cmd")** # Executes the command at the OS level outside R
  Example: >**system("dir")**
  >**system("DATE")**
  >**system("copy con f.bat")**
- **floor(x),ceiling(x),round(x),signif(x),trunc(x)** # Used for rounding
  Examples: >**x=3.75**
  >**floor(x)**
  [1] 3
  >**ceiling(x)**
  [1] 4
  >**round(x)**
  [1] 4

Useful functions in R–Contd.

>**round(x,1)**
[1] 3.8
>**trunc(x)**
[1] 3

- **Sys.time()** # Returns the current system time with the current date.
  >**Sys.time()**
  [1] "2018-08-07 15:44:16 IST"
  Sys.Date() # Returns current system date.
  Example:
  Sys.Date()
  [1] "2018-08-07"
  date() # returns current day,date and time
  >**date()**
  [1] "Tue Aug 07 17:06:27 2018"
- **log(x),log10(x),log2(x),exp(x),sqrt(x)** # Obvious math functions
  Examples: **x=10**
  log(x) #Returns the value of the natural logarithm(to the base e)

Useful functions in R–Contd.

e.g.

>**log(x)**

[1] 2.302585

log10(x) # Returns the value of the logarithm to the base 10

>**log10(x)**

[1] 1

Log2(x) # Returns the value of the logarithm to the base 2

e.g.

>**log2(x)**

[1] 3.321928

exp(x) # Returns the value of e^x

e.g.

>**exp(x)**

[1] 22026.47

e.g.**sqrt(x)** # Returns the value of the square root of x

>**sqrt(x)**

[1] 3.162278

Useful functions in R–Contd.

- **cos(x),sin(x),tan(x),acos(x),asin(x),atan(x)** # Usual trigonometrical functions with x in **radians**.

```
>sin(pi/2)
[1] 1
>sin(pi/4)
[1] 0.7071068
>1/sqrt(2)
[1] 0.7071068
>sin(pi/6)
[1] 0.5
>cos(pi/6)
[1] 0.8660254
>tan(pi/2)
[1] 1.633178e+16
>asin(.5) # inverse function of sin
[1] 0.5235988
>0.5235988*360/(2*pi)
[1] 30
```

- **rep(x,. . .), rep.int(x,times), rep_len(x,length.out)**

Useful functions in R–Contd.

The **rep()** function replicates the values in x, where x may be a vector or a factor, length.out is the desired length of the output vector(a nonegative integer), times refers to a nonnegative integer value implying the number of times to repeat each element if of length length(x) to repeat the whole vector if of length 1. (...), implies ,further arguments to be passed to or from other methods. For the internal default method, these can include times,length.out and each, where each,if mentioned ,will be a nonnegative integer for repeating each of the value of x each times. Other inputs will be coerced to an integer or a double vector and the first element taken. Treated as1 if NA or invalid.

union(),intersect(),setdiff(),setequal() # Usual set operators

Examples:

>**p=c(1,2,3,4,5)** # A vector is defined as a set

>**q=c(3,4,5,6,7,8,9)** # Another set is defined

Now,let us check the functions

>**union(p,q)**

[1] 1 2 3 4 5 6 7 8 9

Useful functions in R–Contd.

```
>intersect(p,q)
[1] 3 4 5
>setdiff(p,q)
[1] 1 2
>setdiff(q,p)
[1] 6 7 8 9
>setequal(p,q)
[1] FALSE
>m=c(1,2,3,4,5)
>setequal(p,m)
[1] TRUE
```

casefold(x, upper = FALSE) By default, **casefold()** converts all characters to lower case, but we can use the **argument upper = TRUE** to indicate the opposite (characters in upper case). Examples:

```
>casefold("ABcdEfghI")
[1] "abcdefghi"
```

Useful functions in R–Contd.

>tolower("ABcdEfghI")
[1] "abcdefghi"
>toupper("ABcdEfghI")
[1] "ABCDEFGHI"
>casefold("ABcdEfghI",upper=TRUE)
[1] "ABCDEFGHI"
chartr(old-char,new-char,vector)
It should be noted that old-char and new-char should have same number of characters;
otherwise, replacement will be done at most in old-char number of characters. It also does single matching character translation.The following example shows this:
chartr("an","the","This is an interesting task")
[1] "This is th ihterestihg ttsk"
However,the number of characters in old-char cannot be more than that in new-char .
>chartr("a","A","That's a boring idea")
[1] "ThAt's A boring ideA"

Useful functions in R–Contd.

```
v=c("a quick brown fox","jumped over","a lazy dog")
>chartr("az","Ad",v)
[1] "A quick brown fox" "jumped over" "A lAdy dog"
chartr("abc","&^%$#","abc represent the first three letters of the English Alphabet")
[1] "&% represent the first three letters of the English Alph&^et"
>chartr("abcd","&^%$#","abcd represent the first four letters of the English Alphabet")
[1] "&^%$ represent the first four letters of the English Alph&^et"
>chartr("abcde","&^%$#","abcde represent the first five letters of the English Alphabet")
[1] "&^%$# r#pr#s#nt th# first fiv# l#tt#rs of th# English Alph&^#t"
>chartr("abcde","^%$#!",
"abcde represent the first five letters of the English Alphabet")
[1] "&^%$# r#pr#s#nt th# first fiv#
l#tt#rs of th English Alph&^#t"
```

Useful functions in R–Contd.

Abbreviate(vector-of-strings,minlength=value,dot=FALSE,method=c(
Here, only the first argument is mandatory and the rest are optional.If only the first argument is supplied, the abbreviation is done by taking the first letter of each syllable(a unit of pronunciation having one vowel sound, with or without surrounding consonants, forming the whole or a part of a word; for example, there are two syllables in water and three in inferno.) of the string in the vector and then shows the string along with the abbreviated form.The following example illustrates this. **v="Damodar Vally Corporation"**
>**abbreviate(v)**
Damodar Vally Corporation
"DmVC"
If we want to abbreviate only with the first letter of each word and to see only the abbreviated form, then the **minlength=1**, specification will be a necessity.

Useful functions in R–Contd.

If we want to put a dot at the end of the abbreviated form, the dot=TRUE is to be mentioned in the function. The method vector includes two options, one of which may be mentioned. The **"left.kept"** option is for the left justification of the of the abbreviated string and the other is for centering. The following figure illustrates the use of all the options one by one.

>abbreviate("Anil Bikash Chowdhury",minlength=1)
Anil Bikash Chowdhury
"ABC"
>abbreviate("Calcutta Electric Supply Corporation", minlength=1,dot=TRUE)
Calcutta Electric Supply Corporation
"CESC."
>v="Damodar Vally Corporation"

Useful functions in R–Contd.

```
>abbreviate(v,minlength=1,method="left.kept")
Damodar Vally Corporation
"DVC"
>cat(abbreviate(v,minlength=1,method="left.kept"))
DVC
>cat(abbreviate(v,minlength=1,method="both.sides"))
DVC
>abbreviate(v)
Damodar Vally Corporation
 "DmVC"
>V=c("United Nations Organizations","World Health Organizations","
Trust of India")
>abbreviate(V,minlength=1,method="left.kept")
United Nations Organizations World Health Organizations
 "UNO"                        "WHO"
 Press Trust of India
"PTol"
```

Useful functions in R–Contd.

```
>AV=abbreviate(V,minlength=1,method="left.kept")
>cat(AV)
UNO WHO PTol
>abbreviate(V,minlength=3)
United Nations Organizations World Health Organizations
 "UNO"                          "WHO"
Press Trust of India
 "PTol"
```

strsplit(string/vector, split=split-value, fixed=FALSE,perl=FALSE, useBytes=FALSE), where,

vector is a character vector, each element of which is to be split. Other inputs, including a factor, will give an error.

Split-value is a character vector (or object which can be coerced to such) containing regular expression(s) (unless fixed = TRUE) to use for splitting. If empty matches occur, in particular if split has length 0, the string or the vector is split into single characters. If split has length is greater than 1, it is re-cycled along the string or vector.fixed is logical. If TRUE match, splitting occurs exactly, otherwise regular expressions is used. It has priority over perl.

Useful functions in R–Contd.

Perl is logical.
useBytes is logical. If TRUE, the matching is done byte-by-byte rather than character-by-character, and inputs with marked encodings are not converted. This is forced (with a warning) if any input is found which is marked as "bytes".
Let us have some illustrative use of the function.Let us reverse a string a string a first: The steps are shown below

```
>string="Education"
>split_string=strsplit(string,split="")
>print(split_string)
[[1]]
[1] "E" "d" "u" "c" "a" "t" "i" "o" "n"
>order=nchar(string):1
>reversed_string=split_string[[1]][order]
>print( reversed_string)
[1] "n" "o" "i" "t" "a" "c" "u" "d" "E"
paste(reversed_string,collapse="")
[1] "noitacudE"
```

So, the given string is reversed in the above process